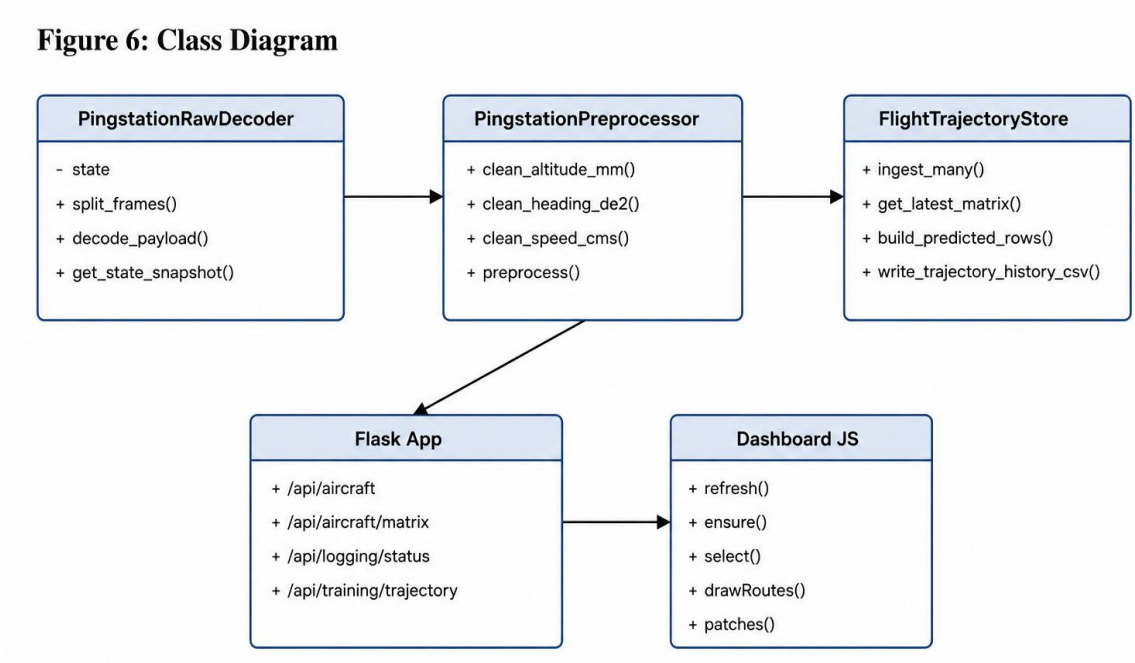


Figure 6: Class Diagram

4.5 Sequence of Operation

At startup, the data-source module reads ADSBTracker.cfg when it is present, then applies environment-variable overrides and built-in defaults. Flask creates shared preprocessing and storage objects, while the data-source module owns the shared decoder used by both replay and live receiver modes. Every second, the browser requests aircraft data; the backend decodes and normalizes the current state, stores eligible samples, adds age-limited predictions for temporarily missing aircraft and returns JSON for the dashboard.

- Application starts, reads effective configuration and prepares output directories.
- Data source selects captured replay or live TCP receiver mode.
- Shared decoder reconstructs frames and merges partial aircraft updates.
- Preprocessor normalizes identity, position, altitude, heading, speed and time fields.
- Storage updates the latest state and writes only threshold-qualified trajectory samples.
- Prediction logic may append labelled output-only rows for recently missing aircraft.
- Flask returns JSON and the dashboard updates markers, tracks, details and charts.